

Symptom: VPN tunnel connects but drops intermittently, especially during high-bandwidth activity (video calls, large transfers)

Overview

The tunnel establishes successfully and works fine for light traffic, but disconnects repeatedly once bandwidth or packet size increases. This pattern points toward MTU/fragmentation issues or a keepalive timeout misconfiguration rather than an authentication or policy problem.

Cause 1: MTU Mismatch / Fragmentation Issues

VPN tunnels add overhead (encryption headers, encapsulation) to every packet, which reduces the effective MTU available for the original payload. If a client or intermediate device sends packets sized for a standard 1500-byte MTU without accounting for this overhead, and Path MTU Discovery is blocked somewhere along the path (a common side effect of ICMP being filtered), oversized packets get silently dropped rather than fragmented — causing exactly the "works for small traffic, fails under load" symptom.

Diagnostic steps:

- Test with a deliberately lowered MTU on the client (e.g., 1400 instead of 1500) to see if the problem disappears — if it does, this confirms an MTU-related cause
- Check whether ICMP (specifically "fragmentation needed" messages) is being blocked anywhere on the path, which would prevent proper Path MTU Discovery
- Consider enabling TCP MSS clamping on the VPN gateway to proactively adjust segment size for tunneled TCP traffic

Cause 2: Dead Peer Detection (DPD) / Keepalive Timeout Misconfiguration

VPN gateways use periodic keepalive messages to detect if the other end of the tunnel is still reachable. If these timers are set too aggressively (too short an interval, too few retries before declaring the peer dead), transient latency spikes — which are more likely during high-bandwidth activity — can cause the gateway to prematurely tear down a tunnel that was actually still fine.

Diagnostic steps:

- Check the configured DPD/keepalive interval and retry count on both ends of the tunnel
- Correlate tunnel drop timestamps against periods of high latency or jitter on the underlying connection

- Consider increasing the keepalive interval or retry threshold if drops correlate with brief latency spikes rather than genuine connectivity loss
-

Symptom: VPN tunnel fails to establish at all — connection attempt times out or is immediately rejected

Overview

Unlike a tunnel that connects and later drops, this is a failure during the initial handshake itself — the client never successfully negotiates a session. This points to the control-plane (IKE/authentication) phase rather than anything related to ongoing data transfer.

Cause 1: IKE Phase 1 Negotiation Failure

Before any user data flows, both ends must agree on encryption/authentication parameters (IKE Phase 1) and establish a secure channel for further negotiation. A mismatch in proposed parameters (encryption algorithm, hash, Diffie-Hellman group, or pre-shared key) between client and gateway causes this phase to fail outright, with no tunnel ever forming.

Diagnostic steps:

- Review IKE negotiation logs on the gateway for a specific rejection reason (proposal mismatch, authentication failure, etc.) rather than assuming from the client-side timeout alone
- Confirm the pre-shared key or certificate configured on the client exactly matches what the gateway expects — a common, easy-to-miss cause
- Verify UDP 500 (and UDP 4500 for NAT-T) aren't being blocked somewhere on the path between client and gateway

Cause 2: NAT Traversal (NAT-T) Not Functioning

When a VPN client sits behind a NAT device (common for remote users on home routers), IKE and ESP traffic need NAT-T to properly traverse that NAT. If NAT-T is disabled, unsupported by an intermediate device, or the required UDP 4500 traffic is blocked, the negotiation can stall or fail silently, particularly for remote-access VPN scenarios.

Diagnostic steps:

- Confirm NAT-T is enabled on both client and gateway configuration
- Check whether UDP 4500 is explicitly allowed through any firewall between the client and gateway, not just UDP 500

- Test from a network without NAT (if possible) to isolate whether NAT traversal is specifically the failure point
-

Symptom: Some applications don't work over VPN even though the tunnel is connected and general connectivity works

Overview

Basic connectivity (ping, simple web browsing) works fine through the tunnel, but specific applications — often ones sending larger packets, like Outlook or file-transfer tools — hang or fail. This is a strong signal of a fragmentation/MSS issue specific to certain traffic patterns, distinct from the general MTU cause above because it's application-selective rather than affecting all high-bandwidth traffic equally.

Cause 1: TCP MSS Not Adjusted for Tunnel Overhead

Applications that negotiate a large TCP Maximum Segment Size during connection setup (based on the client's standard interface MTU, unaware of the VPN's additional overhead) can end up sending packets that are too large for the tunnel to carry without fragmentation — and if fragmentation or Path MTU Discovery isn't working cleanly, those specific application streams hang while smaller-packet traffic (like basic web browsing) continues to work fine.

Diagnostic steps:

- Confirm whether the affected application specifically sends larger payloads than typical web traffic (attachments, database queries, bulk transfers)
- Enable or verify TCP MSS clamping on the VPN gateway, which rewrites the MSS value during the TCP handshake to a tunnel-safe size automatically
- Test the affected application with the client's MTU manually lowered, similar to the general fragmentation diagnostic above, to confirm the same root cause

Cause 2: Split Tunneling Misconfiguration

If split tunneling is enabled (only some traffic routed through the VPN, the rest going direct), and the affected application's traffic is being incorrectly routed — either sent through the tunnel when it shouldn't be, or sent directly when it needs to go through the tunnel to reach an internal resource — it can fail or behave unexpectedly while general connectivity tests (which may hit whichever path happens to be configured correctly) appear fine.

Diagnostic steps:

- Review the split-tunnel routing/ACL configuration to confirm the affected application's destination is correctly included in (or excluded from) the tunneled traffic
 - Use a route trace or the client's local routing table to confirm which path (tunneled vs. direct) is actually being used for the affected application's traffic
-

Symptom: VPN client prompts for credentials repeatedly, or authentication fails even with correct username/password

Overview

The client can reach the VPN gateway (no connection timeout), but the authentication step itself fails or loops. This points to identity verification — certificates, RADIUS/AD backend, or clock sync — rather than network reachability or tunnel negotiation.

Cause 1: Expired or Untrusted Client/Server Certificate

Certificate-based VPN authentication fails outright once either the client's certificate or the gateway's server certificate expires, is revoked, or isn't trusted by the other party's certificate store (e.g., a private CA not yet trusted on new or reimaged client machines).

Diagnostic steps:

- Check certificate expiration dates on both the client and the gateway's presented server certificate
- Confirm the CA that issued the certificates is present and trusted in the client's certificate store, especially on newly provisioned devices
- If the client shows a certificate warning during connection, capture the exact error text — "expired," "untrusted issuer," and "hostname mismatch" all point to different fixes

Cause 2: Backend Authentication Server (RADIUS/AD) Unreachable or Rejecting

When the VPN gateway delegates authentication to an external RADIUS or Active Directory server, a failure or misconfiguration on that backend causes every login attempt to fail or time out, even with entirely correct credentials — the gateway itself may be functioning normally.

Diagnostic steps:

- Check gateway logs for the specific reason authentication failed (timeout reaching RADIUS server vs. an explicit reject response are different problems with different fixes)

- Test connectivity from the VPN gateway to the RADIUS/AD server directly (the gateway needs to reach it, not just the end client)
 - Confirm the shared secret configured between the VPN gateway and RADIUS server hasn't been changed on one side without updating the other
-

Symptom: Always-On VPN client repeatedly reconnects, or fails to reconnect automatically after network changes (e.g., switching from Wi-Fi to cellular)

Overview

Always-On VPN is designed to maintain a persistent tunnel and reconnect automatically as network conditions change. When this reconnection logic breaks down — constant reconnect loops, or a failure to reconnect at all after a network transition — the underlying cause is usually related to how the client detects network changes or validates trust on reconnection, not the VPN tunnel technology itself.

Cause 1: Trusted Network Detection Misconfiguration

Always-On VPN clients typically use "trusted network detection" to decide whether the VPN should even be active (e.g., staying disconnected while on a trusted corporate Wi-Fi network, connecting automatically everywhere else). If this detection is misconfigured — the client incorrectly believes it's on a trusted network when it isn't, or vice versa — it can cause unexpected connect/disconnect cycling as the client repeatedly re-evaluates and flips its decision.

Diagnostic steps:

- Review the trusted network detection criteria configured on the client (DNS suffix match, specific server reachability, or similar) and confirm it accurately reflects the actual trusted network's characteristics
- Check client-side logs for repeated trusted/untrusted network transitions that don't match the user's actual physical movement

Cause 2: Certificate Validation Failing on Network Transition

When a device changes its network path (Wi-Fi to cellular, or between networks), some clients re-validate the server certificate as part of reconnection. If certificate revocation checking (OCSP/CRL) requires reaching an external validation server that isn't reachable from the new network path, the reconnection can hang or fail, even though the underlying tunnel technology itself would otherwise work fine.

Diagnostic steps:

- Check whether certificate revocation checking is enabled on the client, and whether the OCSP/CRL endpoint it depends on is reachable from all expected network paths (including cellular)
- Compare reconnection behavior with revocation checking temporarily disabled, to isolate whether this is the specific cause before deciding on a permanent fix